E — Test-time learning and memory architectures (P6 experimental track: learning while working)

0. Role in the system

This section informs the **P6 experimental track**: the part of the local coding assistant where the model itself changes its weights (or weight-like state) *during use*, not only in the weekly LoRA self-training pass. Five mechanisms are covered: (1) a surprise-written neural memory module whose parameters are updated by gradient descent per token/chunk at inference (Titans, TTT layers); (2) multiple update frequencies inside one model — a continuum from per-token memory through per-task adapters to frozen pre-trained weights (Nested Learning / HOPE); (3) per-task test-time fine-tuning of a LoRA from the task's own few examples with augmentation and voting (ARC test-time training); (4) a model that writes its own fine-tuning data and hyperparameters and is trained by an outer RL/ReST-EM loop to write better ones (SEAL); (5) weight adaptation by tiny per-matrix singular-value "skill vectors" that are RL-trained and mixed at inference by a few-shot search (Transformer^2).

After reading this pack the agents must be able to: implement a small Titans/HOPE-style neural memory in PyTorch (loss, surprise, momentum, forgetting gate, chunked parallel update, MAC/MAG/MAL wiring, CMS multi-frequency scheduling); run per-task test-time training on a 9B worker with QLoRA (task construction, augmentation, loss masking, voting, budgets); implement a SEAL-style self-edit loop driven by the verifier cascade; and implement SVF expert vectors with CEM few-shot mixing as an alternative to a LoRA archive.

1. Papers

1.1 Titans: Learning to Memorize at Test Time (Behrouz et al., 2025) — arXiv:2501.00663

Problem and claim. Recurrent models compress context into a fixed-size state and lose long-range information; attention sees the whole window but costs quadratically. Titans adds a *neural long-term memory*: a small MLP whose weights are updated at test time by gradient descent on an associative-memory loss. The gradient is the "surprise"; a momentum term carries past surprise forward; a data-dependent forgetting gate decays the memory. Attention is framed as short-term memory, the neural memory as long-term memory, and a set of learnable input-independent tokens as persistent (meta) memory. Three ways of wiring memory and attention are given (MAC, MAG, MAL). Claim: Titans beat Transformer++ and modern linear RNNs (Mamba, Mamba2, DeltaNet, Gated DeltaNet, TTT) on language modelling, commonsense reasoning, needle-in-haystack and BABILong, and scale past 2M-token context.

Headline results. (Table 1; FineWeb-Edu; columns Wiki ppl / LMB ppl / Avg acc over 7 commonsense tasks; lower ppl better) - 340M params / 15B tokens: Transformer++ 31.52/41.08/42.92; Mamba 30.83/40.21/43.59; DeltaNet 28.65/47.30/44.04; TTT 27.44/34.19/44.51; Gated DeltaNet 27.01/30.94/45.42; Titans (LMM) 26.18/29.97/46.17; Titans (MAC) 25.43/28.13/47.36; Titans (MAG) 25.07/28.72/47.54; Titans (MAL) 24.69/28.80/46.55. - 400M / 15B: Transformer++ 30.63/37.37/45.64; Mamba 29.22/39.88/45.94; Mamba2 26.34/33.19/46.91; DeltaNet 27.69/44.04/45.57; TTT 26.11/31.52/46.86; Gated DeltaNet 25.47/29.24/47.26; LMM 25.03/28.99/47.83; MAC 25.61/27.73/48.65; MAG 23.59/27.81/48.60; MAL 23.93/27.89/47.87. - 760M / 30B: Transformer++ 25.21/27.64/48.69; Mamba 28.12/23.96/47.22; Mamba2 22.94/28.37/48.34; DeltaNet 24.37/24.60/48.97; TTT 24.17/23.51/47.32; Gated DeltaNet 21.18/22.09/49.69; LMM 20.04/21.96/51.56; MAC 19.93/20.12/52.51; MAG 18.61/19.86/52.50; MAL 19.07/20.33/50.97. - S-NIAH (RULER, Table 2; accuracy % at 2K/4K/8K/16K). S-NIAH-PK: TTT 98.4/98.8/98.0/88.4; Mamba2 98.6/61.4/31.0/5.4; DeltaNet 96.8/98.8/98.6/71.4; LMM 99.8/98.4/98.2/96.2; MAC 99.2/98.8/99.0/98.4; MAG 99.4/98.0/97.4/97.4; MAL 98.8/98.6/98.8/97.8. S-NIAH-N: TTT 60.2/36.6/10.2/4.4; Mamba2 98.4/55.8/14.2/0.0; DeltaNet 47.2/15.4/12.8/5.4; LMM 100.0/99.8/93.4/80.2; MAC 99.6/98.2/97.6/97.4; MAG 99.2/98.8/97.2/98.6; MAL 99.8/98.1/96.8/96.4. S-NIAH-W: TTT 78.8/28.0/4.4/0.0; Mamba2 42.2/4.2/0.0/0.0; DeltaNet 46.2/20.0/1.6/0.0; LMM 90.4/89.4/85.8/80.6; MAC

98.2/98.2/95.6/95.2; MAG 98.0/98.0/90.2/88.2; MAL 98.0/97.4/92.0/90.4. - BABILong (Section 5.4): text claims fine-tuned Titans (MAC) outperform GPT-4, GPT4o-mini, Qwen2.5-72B, Llama3.1-70B, RMT and Llama3.1-8B+RAG (the latter with about 70x fewer parameters); exact accuracies are only in Figure 6 (not in fetched text — verify in paper §5.4).

Method — equations. (Section 3.1 unless noted; x_t is the input at step t, $\mathcal{M}_t$ the memory parameters.)

Basic surprise update, Eq. (8):

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \theta_t \nabla\, \ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right)$$

θ_t: data-dependent step size; $\nabla\, \ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right)$: gradient of the memory loss = the "surprise".

Past + momentary surprise, Eqs. (9)–(10):

$$\mathcal{M}_t = \mathcal{M}_{t-1} + S_t, \qquad S_t = \eta_t S_{t-1} - \theta_t \nabla\, \ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right)$$

S_t: surprise/momentum state; $\eta_t S_{t-1}$: past surprise with data-dependent decay η_t; $\theta_t \nabla\, \ell$: momentary surprise.

Keys/values, Eq. (11): $\mathbf{k}_t = x_t W_K$, $\mathbf{v}_t = x_t W_V$, with $W_K, W_V \in \mathbb{R}^{d_{in} \times d_{in}}$ learned in the outer loop.

Associative memory loss, Eq. (12):

$$\ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right) \,=\, \left\| \, \mathcal{M}_{t-1}\, \left(\, \mathbf{k}_t \,\right) - \mathbf{v}_t \, \right\|_2^2$$

Update with forgetting gate, Eqs. (13)–(14):

$$\mathcal{M}_t = \, \left(\, 1 - \alpha_t \,\right) \mathcal{M}_{t-1} + S_t, \qquad S_t = \eta_t S_{t-1} - \theta_t \nabla\, \ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right)$$

$\alpha_t \in \, \left[\, 0, 1 \,\right]$: forgetting gate (weight decay); $\alpha_t \to 0$ keeps memory, $\alpha_t \to 1$ clears it.

Linear case (text): $\mathcal{M} = W \in \mathbb{R}^{d_{in} \times d_{in}}$, $\ell\, \left(\, W_{t-1}; x_t \,\right) \,=\, \| \, W_{t-1} \mathbf{k}_t - \mathbf{v}_t \, \|_2^2$ is online linear regression; the paper uses an MLP memory with $L_{\mathcal{M}} \geq 1$ layers and argues $L_{\mathcal{M}} \geq 2$ is strictly more expressive.

Retrieval, Eq. (15):

$$y_t = \mathcal{M}^*\, \left(\, \mathbf{q}_t \,\right), \qquad \mathbf{q}_t = x_t W_Q$$

$\mathcal{M}^*$: forward pass without weight update.

Chunked parallel form (Section 3.2), Eq. (16), chunk size $b \geq 1$:

$$\mathcal{M}_t = \, \left(\, 1 - \alpha_t \,\right) \mathcal{M}_{t-1} - \theta_t \nabla\, \ell\, \left(\, \mathcal{M}_{t-1}; x_t \,\right) \,= \beta_t \mathcal{M}_0 - \sum_{i\,=\,1}^{t} \theta_i \frac{\beta_t}{\beta_i} \nabla\, \ell\, \left(\, \mathcal{M}_{t'}; x_i \,\right)$$

$$t' = t - \mathrm{mod}\, \left(\, t, b \,\right), \qquad \beta_{\,i} = \prod_{j\,=\,1}^{i} \left(\, 1 - \alpha_j \,\right)$$

All gradients in a chunk are taken at the chunk-start state $\mathcal{M}_{t'}$. Eq. (17), linear memory, first chunk:

$$\sum_{i\,=\,1}^{b} \theta_i \frac{\beta_b}{\beta_i} \nabla\, \ell\, \left(\, W_0; x_i \,\right) \,= \Theta_b \mathbf{B}_b \, \left(\, W_0 X - X \,\right) X^\top, \qquad \Theta_b = \mathrm{diag}\, \left(\, \theta_1, ..., \theta_b \,\right)$$

($\mathbf{B}_b$ defined analogously from β_b / β_i; the paper stores Θ_{kb}, $\mathbf{B}_{kb}$ per chunk.) Momentum as a linear recurrence, Eq. (18): $S_t = \eta_t S_{t-1} - \theta_t u_t$, $u_t = \nabla\, \ell\, \left(\, \mathcal{M}_{t'}; x_t \,\right)$, computed with a parallel associative scan. Making α, θ, η functions of the chunk instead of the token gives an LTI system computable by global convolution (less expressive; left to future work).

Persistent memory (Section 3.3), Eq. (19): $P = \left[p_1 \, p_2 \, ... \, p_{N_p} \right]$, $N_p \geq 1$ learnable input-independent vectors; $x_{new} = P \parallel x$ (prepend).

MAC (Section 4.1): split input into segments $S^{(i)}$ of length C; Eqs. (21)–(25):

$$\mathbf{h}_t = \mathcal{M}^*_{t-1} \left(\, \mathbf{q}_t \,\right), \; \mathbf{q}_t = S^{(t)} W_Q; \quad \tilde{S}^{(t)} = P \parallel \mathbf{h}_t \parallel S^{(t)}; \quad y_t = \mathrm{Attn} \left(\tilde{S}^{(t)} \right); \quad \mathcal{M}_t = \mathcal{M}_{t-1} \left(\, y_t \,\right); \quad o_t = y_t \otimes \mathcal{M}^*_t \left(\, y_t \,\right)$$

($\mathcal{M}_{t-1} \left(\, y_t \,\right)$ = memory updated by Eqs. 13–14 on y_t; $\otimes$ = gating combination.)

MAG (Section 4.2), Eqs. (26)–(28): $\tilde{x} = P \parallel x$; $y = \text{SW-Attn}^*\, \left(\, \tilde{x} \,\right)$; $o = y \otimes \mathcal{M}\, \left(\, \tilde{x} \,\right)$, with SW-Attn^* sliding-window attention with prefix, and $\otimes$ a non-linear gate: both branches normalised with learnable vector weights then combined through $\sigma\, \left(\, \cdot \,\right)$.

MAL (Section 4.3), Eqs. (29)–(31): $\tilde{x} = P \parallel x$; $y = \mathcal{M}(\tilde{x})$; $o = \text{SW-Attn}(y)$. “Memory Without Attention” = LMM alone = “Titans (LMM)”.

Architectural details (Section 4.4): residual connections everywhere; SiLU on Q/K/V computation; ℓ_2 -normalised queries and keys; 1D depthwise-separable convolution after each Q/K/V projection; normalisation + linear gate before the output projection.

Method — algorithm. (Derived directly from Eqs. 11–15, one head; $b = 1$)

```
state: M (MLP weights, L_M layers), S (same shape as M, init 0)
for each token x_t:
    k = x_t W_K ; v = x_t W_V ; q = x_t W_Q
    g = grad_M || M(k) - v ||^2            # surprise, autograd w.r.t. M
    S = eta_t * S - theta_t * g             # past + momentary surprise
    M = (1 - alpha_t) * M + S               # forget, then write
    y_t = M(q)                              # retrieval (no update)
```

Chunked ($b > 1$): compute all g_i in the chunk at $\mathcal{M}_{t'}$ with one batched forward/backward (Eq. 16–17), run the scan of Eq. (18) for S, then apply the per-token decays β_t/β_i and add. The functional form of $\alpha_t, \theta_t, \eta_t$ is only described as “functions of x_t” (not in fetched text — verify in paper §3.1).

Hyperparameters and setup. FineWeb-Edu; LLaMA-2 tokenizer, 32K vocab; 4K training context; AdamW, lr 4e-4, cosine annealing, batch 0.5M tokens, weight decay 0.1; sizes 170M/340M/400M (15B tokens) and 760M (30B tokens). Memory depth $L_{\mathcal{M}} \in \{1, 2, 3, 4\}$ tested in Section 5.5 (deeper = better perplexity at all lengths, more robust at small size; throughput falls linearly with depth). Chunk size b, segment size C, window size, N_p, memory hidden width: (not in fetched text — verify in paper §5.1 / appendix).

Ablations that matter. (Table 5, Section 5.9; LM ppl / Reasoning acc / Long-context acc) LMM 27.01/47.83/92.68; +Attn (MAC) 26.67/48.65/97.95; +Attn (MAG) 25.70/48.60/96.70; +Attn (MAL) 25.91/47.87/96.91; Linear memory 28.49/46.97/85.34; w/o convolution 28.73/45.82/90.28; w/o momentum 28.98/45.49/87.12; w/o weight decay (forgetting) 29.04/45.11/85.60; w/o persistent memory 27.63/46.35/92.49. Ranked importance: forgetting gate > momentum > deep (non-linear) memory > convolution > persistent memory.

Code / data / license. No official code at fetch time (“intend to make the code we used to train and evaluate our models available soon”; implemented in PyTorch and JAX). Unofficial PyTorch: https://github.com/lucidrains/titans-pytorch (MIT; `NeuralMemory(dim, chunk_size)`, `MemoryAsContextTransformer`). Data: FineWeb-Edu. Paper license not stated.

Stated limitations. No dedicated section. The memory is a simple MLP; better memory architectures left to future work. Chunk-level (not token-level) gates trade expressiveness for speed. Deeper memory trains slower. Larger-model results promised for a later version.

Use in our design. The neural memory is the P6 “learns while working” primitive: a 2-layer MLP memory per head with surprise + momentum + forgetting, updated by chunk during the worker’s forward pass. For a single-user local system: keep the memory state per project (serialise $\mathcal{M}, S$ to disk at session end), start with MAG wiring around a frozen backbone (the memory branch can be trained alone with the backbone frozen), chunk size tied to the serving batch, and gate the whole branch off by default so the production path is unaffected. Use the ablation ranking to prioritise: implement forgetting and momentum before anything else.

PDF: https://arxiv.org/pdf/2501.00663 — Code (unofficial): https://github.com/lucidrains/titans-pytorch

1.2 Learning to (Learn at Test Time): RNNs with Expressive Hidden States (Sun et al., 2024) — arXiv:2407.04620

Problem and claim. RNN hidden states are fixed-size vectors/matrices and plateau in long context; attention keeps improving but is quadratic. The paper makes the hidden state *itself a model* f with weights W, updated at every token by one step of gradient descent on a self-supervised loss computed from the token — even at test time (“TTT layers”). TTT-Linear (f linear) and TTT-MLP (f two-layer MLP) match or beat Transformer and Mamba at 125M–1.3B, and unlike Mamba keep improving past 16k context.

Headline results. Exact per-size perplexities are only in Figures 10, 11, 15 (not in fetched text — verify in paper §3). Text: on the Pile at 2k, TTT-Linear, Mamba and Transformer are comparable; at 8k both TTT variants beat Mamba; on Books at 32k both TTT variants beat Mamba; Mamba’s perplexity plateaus after 16k tokens while TTT and Transformer keep improving (Figure 2). Matched-FLOPs 125M TTT-Linear perplexity 11.09 (Table 1). Latency per token on A100 (Figure 12): Transformer grows linearly with context, TTT-Linear/TTT-MLP/Mamba

roughly constant. Training on TPU v5e-256 at 2k: TTT-Linear 0.27 s/iter vs Transformer 0.30 s/iter. Dual form over 5x faster than primal (JAX, TPU).

Method — equations. (Section 2)

Output and update rules, Eqs. (1)–(2):

$$z_t = f\left(x_t; W_t\right), \qquad W_t = W_{t-1} - \eta \nabla \ell\left(W_{t-1}; x_t\right)$$

z_t output token; f inner-loop model; W_t hidden state (= weights of f); η inner learning rate; ℓ self-supervised loss.

Naive reconstruction, Eq. (3): $\ell\left(W; x_t\right) = \left\| f\left(\tilde{x}_t; W\right) - x_t \right\|^2$ with $\tilde{x}_t$ a corrupted x_t.

Multi-view reconstruction (Section 2.3), Eqs. (4)–(5):

$$\ell\left(W; x_t\right) = \left\| f\left(\theta_K x_t; W\right) - \theta_V x_t \right\|^2, \qquad z_t = f\left(\theta_Q x_t; W_t\right)$$

θ_K training view, θ_V label view, θ_Q test view — all outer-loop parameters; only W is updated in the inner loop.

Mini-batch TTT (Section 2.4), Eq. (6): $W_t = W_{t-1} - \eta G_t = W_0 - \eta \sum_{s=1}^{t} G_s$, with $G_t = \nabla \ell\left(W_{t'}; x_t\right)$, $t' = t - \mathrm{mod}\left(t, b\right)$, b = TTT mini-batch size (paper: $b = 16$). All gradients of a mini-batch are taken at the same $W_{t'}$, so they are computed in parallel; W_t still follows the cumulative sum.

Dual form (Section 2.5; TTT-Linear, $\theta_K = \theta_V = \theta_Q = I$, first mini-batch, $X = \left[x_1, ..., x_b\right]$):

$$G_t = 2\left(W_0 x_t - x_t\right) x_t^\top, \qquad W_b = W_0 - 2\eta\left(W_0 X - X\right) X^\top$$

Eq. (7): $z_t = W_0 x_t - 2\eta \sum_{s=1}^{t} \left(W_0 x_s - x_s\right) x_s^\top x_t$. Eq. (8): $\Delta = \left(W_0 X - X\right) \mathrm{mask}\left(X^\top X\right)$ (upper-triangular zero mask so $\delta_t = \sum_{s \le t} \cdots$), and $Z = W_0 X - 2\eta\Delta$. General case (Appendix A, $\eta = 1$): with $\widehat{X} = \theta_K X$, $Y = \theta_V X$, $\bar{X} = \theta_Q X$, forward $Z^k = W_0^k \widehat{X}^k$, $\widehat{X}^{k+1} = \sigma_k\left(Z^k\right)$; backward $\nabla_{Z^k} \ell = {\sigma_k}'\left(Z^k\right) \odot \nabla_{\widehat{X}^{k+1}} \ell$, $\nabla_{\bar{X}^k} \ell = \left(W_0^k\right)^\top \nabla_{Z^k} \ell$, $\nabla_{W_0^k} \ell = \nabla_{Z^k} \ell \left(\widehat{X}^k\right)^\top$, $W_b^k = W_0^k - \nabla_{W_0^k} \ell$; output $\bar{Z}^k = W_0^k \bar{X}^k - \nabla_{Z^k} \ell \cdot \mathrm{mask}\left(\left(\widehat{X}^k\right)^\top \bar{X}^k\right)$, $\bar{X}^{k+1} = \sigma_k\left(\bar{Z}^k\right)$.

Theorem 1 (Section 2.6): TTT-Linear with batch GD ($b = T$), $\eta = 1/2$, $W_0 = 0$ equals linear attention: $W_t = \sum_{s \le t} \left(\theta_V x_s\right) \left(\theta_K x_s\right)^\top$, $z_t = \sum_{s \le t} \left(\theta_V x_s\right) \left(\theta_K x_s\right)^\top \left(\theta_Q x_t\right)$. Theorem 2: the Nadaraya–Watson estimator with kernel $\kappa\left(x, x'\right) \propto \exp\left(\left(\theta_K x\right)^\top \theta_Q x'\right)$ equals self-attention.

Implementation (Section 2.7): $f_{lin}\left(x\right) = Wx$; TTT-MLP = two-layer MLP, hidden $4\times$ input, GELU; both wrapped as $f\left(x\right) = x + \mathrm{LN}\left(f_{res}\left(x\right)\right)$; learnable per-token inner lr

$$\eta\left(x\right) = \eta_{base}\, \sigma\left(\theta_{lr} \cdot x\right)$$

(θ_{lr} learnable vector, σ sigmoid; $\eta_{base} = 1$ for TTT-Linear, 0.1 for TTT-MLP).

Method — algorithm.

```
outer loop: next-token loss over the whole network; params = backbone, theta_K, theta_V, theta_Q, theta_lr,
W_0 (learnable);
            backprop THROUGH the inner updates (gradients of gradients), gradient checkpointing through
time.
inner loop (one TTT layer, one sequence):
  W <- W_0
  for each mini-batch X_c of b=16 tokens:
      Xk = theta_K X_c ; Xv = theta_V X_c ; Xq = theta_Q X_c ; eta_c = eta_base * sigmoid(theta_lr . X_c)
      # dual form: one forward/backward at W gives all per-token gradients and the causal intra-batch
outputs
      G = grad_W sum_t || f(Xk_t; W) - Xv_t ||^2   (per-token, taken at the same W)
      Z_c = f(Xq; W) - eta_c-weighted causal cumulative correction (Eq. 8 / Appendix A)
      W <- W - sum_t eta_t G_t
```

Hyperparameters and setup. Backbones: Transformer backbone (TTT layer replaces self-attention, NormFormer LN before output projection) and Mamba backbone (temporal conv before the sequence layer, GELU, gate; θ_K and θ_Q merged). Sizes: 125M (12 blocks, 768, 12 heads), 350M (24, 1024, 16), 760M (24, 1536, 16), 1.3B (24, 2048, 32). Chinchilla recipe from the Mamba paper: AdamW $\beta = \left(0.9, 0.95\right)$, wd 0.1, cosine to 1e-5, 10% linear warmup, clip 1.0, no dropout, batch 0.5M tokens, Llama tokenizer; peak lr / steps / tokens: 3e-3/4800/2.5B (125M), 1.5e-3/13500/7B (350M), 1.25e-3/29000/15B (760M), 1e-3/50000/26B (1.3B). Data: Pile at

2k and 8k; Books3 1k–32k. $b = 16$; $\eta_{base} \in \{0.01, 0.1, 1, 10\}$ swept, largest stable kept; TTT-MLP warms up η_{base} over 10% of steps. Transformer long-context fine-tune: RoPE $\theta = 500{,}000$.

Ablations that matter. (Table 1, 125M; perplexity, delta) original linear attention 15.91; improved linear attention 15.23 (−0.68); TTT equivalence 15.23 (0); learnable W_0 15.27 (+0.04, but required for stability of later rows); LN + residual in f 14.05 (−1.22); mini-batch TTT $b = 16$ 12.35 (−1.70); learnable η 11.99 (−0.36); Mamba backbone 11.09 (−0.90). Smaller b = more gradient steps = better perplexity; $b = 16$ chosen for speed/quality. Convolutions help more when the hidden state is less expressive (TTT-MLP gains less from the Mamba backbone).

Code / data / license. https://github.com/test-time-training/ttt-lm-pytorch (official, MIT, "naive tutorial implementation", `ttt.py`, 1B-style default config); https://github.com/test-time-training/ttt-lm-jax (training/reproduction); `ttt-lm-kernels` (inference kernels). Data: Pile, Books3.

Stated limitations. TTT-MLP wall-clock cost exceeds its FLOP advantage (memory I/O); only inference kernels written for GPU; no hybrid attention+TTT tested; not trained at millions of tokens or billions of parameters. Future work explicitly names "multi-level nested learning" if f is itself a self-attention layer.

Use in our design. This paper supplies the *engineering* of Titans: the mini-batch dual form is how the per-chunk memory update is computed with matmuls instead of a per-token autograd loop; LN + residual inside f and learnable $\eta\,(\,x\,)$ are the two stability measures to copy verbatim; $b = 16$ is the starting chunk size. For a single-user system, use the PyTorch tutorial implementation as the reference for a from-scratch `TTTLinear` module, then add Titans' momentum and forgetting on top.

PDF: https://arxiv.org/pdf/2407.04620 — Code: https://github.com/test-time-training/ttt-lm-pytorch

1.3 Self-Adapting Language Models (SEAL) (Zweiger et al., 2025) — arXiv:2506.10943

Problem and claim. LLM weights are static; adapting to new knowledge or tasks uses fixed, hand-designed data pipelines. SEAL makes the model generate its own fine-tuning data and update directives ("self-edits"), applies them with supervised fine-tuning (LoRA) as a persistent weight change, and trains the self-edit *policy* with reinforcement learning whose reward is the downstream performance of the updated model. The outer loop is ReST-EM (rejection sampling + SFT on rewarded self-edits), because the reward depends on the current weights and must be on-policy.

Headline results. Few-shot ARC (Table 1; Llama-3.2-1B-Instruct; 8 held-out tasks x 5 self-edits = 40; success = % of self-edits whose adapted model solves the held-out test): ICL 0%; TTT + self-edit without RL 20%; SEAL 72.5%; Oracle TTT (human-chosen config) 100%. Knowledge incorporation (Table 2; Qwen2.5-7B; SQuAD, no passage in context; accuracy %; columns single passage n=1 (LoRA) / continued pretraining n=200 (full FT) / n=2067 (full FT)): Base 32.7/32.7/29.0; Train on passage 33.5/36.0/31.2; Passage + self-generated implications (no RL) 39.7/50.6/43.4; Passage + GPT-4.1 implications 46.3/59.4/49.2; SEAL 47.0/58.2/46.4. SEAL surpasses GPT-4.1-generated data after 2 ReST-EM rounds on single-passage; later rounds give diminishing returns (Figure 4; per-round values not in fetched text).

Method — equations. (Section 2)

RL objective, Eq. (1):

$$\mathcal{L}_{RL}\left(\theta_t\right) := -\mathbb{E}_{(C,\tau)\sim\mathcal{D}}\left[\mathbb{E}_{SE\sim \mathrm{LM}_{\theta_t}(\,\cdot\mid C)}\left[\,r\left(SE,\tau,\theta_t\right)\,\right]\right]$$

θ_t current LM parameters; C context (passage or few-shot demos); τ downstream evaluation (questions+answers or test input+output); SE self-edit (generated token sequence: data and/or hyperparameters); $\mathcal{D}$ dataset of $(\,C,\tau\,)$.

Reward, Eq. (2):

$$r\left(SE,\tau,\theta_t\right) = \begin{cases} 1 & \text{if adaptation with } SE \text{ improves } \mathrm{LM}_{\theta_t}\text{'s performance on } \tau \\ 0 & \text{otherwise} \end{cases}$$

where adaptation is $\theta_t' = \mathrm{SFT}\left(\theta_t, SE\right)$ (LoRA).

Policy gradient (Eqs. 3–4; N contexts, M self-edits each, T tokens per self-edit, reward treated as constant w.r.t. θ_t):

$$\nabla_{\theta_t}\mathcal{L}_{RL} \approx -\frac{1}{NM}\sum_{i=1}^{N}\sum_{j=1}^{M} r_{ij}\sum_{s=1}^{T}\nabla_{\theta_t}\log p_{\theta_t}\left(y_s^{(i,j)} \mid y_{<s}^{(i,j)}, C_i\right)$$

With binary r, this is SFT on the self-edits that earned reward 1 (filtered behaviour cloning / ReST-EM E-step: sample from current policy; M-step: SFT on rewarded samples). GRPO and PPO were tried and were unstable.

Method — algorithm. (Algorithm 1 as printed, single sample per iteration; the text batches N contexts x M samples)

```
Input: LM_theta, dataset D = {(C, tau)}
for outer iteration t = 1, 2, ...:
    sample (C, tau) ~ D
    SE ~ LM_theta_t(. | C)                    # generate self-edit
    theta_t' <- SFT(theta_t, SE)              # inner-loop update (LoRA)
    Ans ~ LM_theta_t'(. | tau)                # evaluate adapted model
    r <- r(Ans, tau)                          # binary reward
    theta_{t+1} <- RL_Update(theta_t, r, SE)  # ReST-EM: SFT on SEs with r = 1
```

Knowledge incorporation: self-edit = "List several implications derived from the content" of the passage; inner SFT = causal LM loss on the self-edit sequence with LoRA; reward = accuracy on the passage's questions without the passage in context; reward may be assigned only to the best self-edit among the M samples. Few-shot ARC: self-edit = JSON-like spec of tools — booleans for basic / size / chain / repeat augmentations, learning rate (float), epochs (int), and loss strategy (all tokens vs output tokens); the spec drives a TTT run (per the ARC-TTT recipe, paper 1.4); reward 1 iff the adapted model outputs the correct held-out test grid.

Hyperparameters and setup. ARC (Appendix A): tasks filtered to those solvable under an optimal TTT config; 11 training tasks, 8 held-out; $M = 15$ self-edits per training task; one ReST-EM round; inner TTT LoRA rank 128, $\alpha = 16$ (lr/epochs chosen by the self-edit; example 1e-5, 3 epochs); Oracle TTT: rank 128, α 16, 2 epochs, batch 2, lr 1e-4; ReST-EM SFT on good self-edits: rank 16, α 16, 8 epochs, batch 5, lr 5e-5. Knowledge incorporation (Appendix B): $N = 50$ contexts x $M = 5$ self-edits at temperature 1 per round, 2 rounds, 3 seeds; inner LoRA search space (Table 3, single passage): rank {32,64}, α {32,64}, lr {1e-4,3e-4,5e-4,1e-3,2e-3}, epochs {1,5,10,15,20}, batch {1,4}; multi-passage (Table 4): epochs {1,3,5}, batch {1,4,8,16}; ReST-EM SFT: LoRA on all MLP and attention projections, rank 64, α 128, batch 10, 2 epochs, lr 3e-4; grader GPT-4.1 (gpt-4.1-2025-04-14, greedy). Continued pretraining uses 5 self-edit generations per passage, full fine-tuning (hyperparameters not in fetched text).

Ablations that matter. Self-edits without RL already help (20% vs 0% ICL on ARC; 39.7 vs 33.5 on SQuAD); RL adds +52.5 points on ARC and +7.3 points on single-passage SQuAD. RL makes self-edits longer/more detailed and converges to "atomic fact" style (Figure 5). Oracle TTT = 100% shows the ceiling is the self-edit policy, not the TTT machinery.

Code / data / license. https://github.com/Continual-Intelligence/SEAL (MIT; folders `few-shot/`, `general-knowledge/`; Python 3.12; "all experiments can run on 2 A100/H100 GPUs"; needs `OPENAI_API_KEY` for the GPT-4.1 grader). Project page https://jyopari.github.io/posts/seal. Data: SQuAD, ARC.

Stated limitations. (Section 5) Catastrophic forgetting: with sequential self-edits on new passages, accuracy on earlier passages "gradually declines as the number of edits increases" though "without complete collapse" (Figure 6; numbers not in fetched text — verify in paper §5 / Appendix B.6 Table 5). Compute: "each self-edit evaluation takes approximately 30–45 seconds" (a full fine-tune + eval per sample). Context-dependent evaluation: every context needs paired labelled τ, "prevents RL training of SEAL from scaling to unlabeled corpora"; proposed fix: let the model write its own evaluation questions.

Use in our design. SEAL is the outer loop of the weekly LoRA self-training module *and* the P6 per-task self-edit: the verifier cascade (tests, type-check, lint) is the binary reward r, the self-edit is "synthetic training examples + LoRA hyperparameters" the worker writes for a task class, and ReST-EM (filter rewarded edits, SFT) is the update. For a single user: keep M small (3–5), run the inner SFT with QLoRA on the 9B worker, archive each rewarded self-edit as a reusable dataset, and never apply sequential edits to one weight set — the forgetting result argues for per-topic LoRA pockets with a frozen base.

PDF: https://arxiv.org/pdf/2506.10943 — Code: https://github.com/Continual-Intelligence/SEAL

1.4 The Surprising Effectiveness of Test-Time Training for Few-Shot Learning (Akyürek et al., 2024) — arXiv:2411.07279

(v1 title: "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning"; v2, March 2025, adds BIG-Bench Hard.)

Problem and claim. LMs fail on structurally novel tasks even with in-context examples. Test-time training (TTT) — a temporary LoRA update per test task, trained on data built from that task's own demonstrations — raises an 8B model to 53.0% on the ARC public validation set (up to 6x the fine-tuned baseline) and 61.9% when ensembled with BARC program synthesis, matching average human performance (60.2%). On BBH, TTT on 10-shot examples beats 10-shot prompting by 7.3 points (50.5% to 57.8%).

Headline results. ARC public evaluation (Table 1, pass@2, exact match on all test outputs): fine-tuned 8B 18.3% (Table: 18.25%) → + TTT 47.1% (47.125%); BARC neural model + this TTT 53.0%; + BARC synthesizer 58.5%; full ensemble 61.9% in text (Table prints 62.8 — source inconsistent; verify in paper Table 1); BARC original ensemble 54.375%, BARC without synthesizer 39.25%; humans average 60.2%, best 97.8%; semi-private ARC-AGI 47.5%. Tasks solved out of the 80-task balanced validation subset (Figure 13): 1B FT 5 → FT+TTT 29; 3B 11 → 29; 8B 14 → 36. Program-synthesis overlap: BARC neural solves 42.2% of tasks solved by program synthesis; with TTT 73.5%. BBH (Llama-3.1-8B, 5 seeds): zero-shot 40.9%, 10-shot ICL 50.5%, TTT 57.8%; Dyck languages and Ruin names gain roughly 20–50 points.

Method — equations. (Section 2.2 and 3)

Per-task objective: starting from θ_0, build $\mathcal{D}_{TTT}$ from the task and set

$$\theta_d = \arg\min_{\theta} \sum_{d_{TTT} \in \mathcal{D}_{TTT}} \mathcal{L}(\mathrm{LM}(d_{TTT}; \theta))$$

then predict the task's test input with θ_d.

Leave-one-out task construction (Section 3.1), given K demonstration pairs $\{(x_k, y_k)\}_{k=1}^{K}$:

$$d_j^{ICL} = (\{(x_k, y_k)\}_{k \neq j}, x_j, y_j), \quad j = 1, ..., K$$

plus permutations of demonstration order; the direct-I/O alternative $d_j^{I/O} = (x_j, y_j)$ underperforms. Augmentation with a set $\mathcal{T}$ of invertible transformations ($t^{-1}(t(x)) = x$):

$$\mathcal{D}_{TTT} = \bigcup_{t \in \mathcal{T}} \bigcup_{j} t(d_j)$$

Loss variants (Section 3.2):

$$\mathcal{L}^{label} = \mathcal{L}_{LM}(y_{test} \mid x_1, y_1, ..., x_K, y_K, x_{test}; \theta)$$

$$\mathcal{L}^{outputs} = \mathcal{L}^{label} + \sum_{k=2}^{K} \mathcal{L}_{LM}(y_k \mid x_1, y_1, ..., x_k; \theta)$$

$$\mathcal{L}^{all} = \mathcal{L}^{outputs} + \sum_{k} \mathcal{L}_{LM}(x_k \mid x_1, y_1, ..., y_{k-1}; \theta)$$

($\mathcal{L}^{outputs}$ — loss on every demonstration output plus the held-out output — works best; sum starts at $k = 2$ for ARC.)

Hierarchical voting (Section 4.2 / Appendix E): per transformation t with prediction multiset P_t, $S_t = \mathrm{Top}_3(P_t)$; global $\widehat{Y} = \mathrm{Top}_2(\bigcup_t S_t)$.

Method — algorithm. (ARC recipe, as stated)

```
Input: task (K demos, x_test), base theta_0 (already fine-tuned on ARC-like data), transforms T
1. for j in 1..K: d_j = (demos without j, x_j, y_j)                    # leave-one-out
2. D_TTT = { t(d_j) : t in T, j }  with demo-order permutations and colour shuffles; cap 250 examples
3. train ONE LoRA per task on D_TTT with L^outputs (rank 128, alpha 16, 2 epochs, bs 1-2, AdamW,
   lr 5e-5 for 1B/3B, 1e-4 for 8B; modules: attention q,v projections, MLP weights, output projection)
4. inference: for each t in T, for 2 sampled demo orders: y = t^-1( greedy LM_{theta_0 + LoRA}( t(demos),
t(x_test) ) )
5. per t: S_t = top-3 most frequent (if < 3 unique, add row-wise and column-wise majority candidates)
   global: top-2 over the union; ties -> identity-transform predictions first
6. submit the 2 candidates (pass@2); discard the LoRA
```

Transformations used in TTT: 90° and 180° rotations, horizontal and vertical flips, transpose; plus colour shuffling and demonstration-order shuffling (Table 5 lists more, e.g. 270° rotation, reflections, random translation, resolution increase; which were used is not stated).

Hyperparameters and setup. Models: Llama-3.2-1B (ablations), Llama-3.2-3B, Llama-3-8B-Instruct (final); BBH on Llama-3.1-8B. Pre-TTT fine-tuning (Appendix B, full FT with torchtune): lr 2.5e-5, 2 epochs (up to 16,000

steps), batch 32, AdamW, cosine with 2k warmup; data = 6,426 LM-generated task generators (GPT-4/GPT-4o) + REARC + rule-based augmentations; 2xA100 (1B), 4xA100 (3B/8B). TTT (Table 6): above. Validation subset: 80 tasks, 20 each easy/medium/hard/expert. BBH (Table 8): LoRA rank 64, 40 random demonstration shuffles, search rank {64,128}, α {16,32,64,128}, lr {1e-5,5e-5,1e-4,3e-4}, 1 epoch, batch 5, 20–60 steps, AdamW cosine; 240 eval examples per task; no transformations, greedy. Compute: ARC TTT about 12 hours on one A100 for 100 validation tasks (about 7 min/task); BBH about 15 minutes on one A100 for full TTT + inference.

Ablations that matter. (1B, 80 tasks, Section 4.3) FT baseline 5 tasks → full TTT 29 (about 6x). Direct I/O instead of leave-one-out ICL: −11 tasks (38% relative); no transformation augmentation: −16 tasks (55% relative); shared LoRA across tasks instead of per-task: −7 tasks (24% relative); dropping demonstration-output loss: 29% → 26%. Voting (Section 4.5, Figure 7): vanilla < single transformation < flat voting < hierarchical voting ≈ oracle (numbers not in fetched text). Shared adapter hurts ARC but helps BBH (Section 5.3).

Code / data / license. ARC: https://github.com/ekinakyurek/marc; BBH: https://github.com/adamzweiger/Fewshot-TTT. Code license not stated. Data: ARC, REARC, BARC, BBH.

Stated limitations. Section headings “Optimization bias” and “Data leakage” (text not in fetched HTML — verify in paper §6). Result inconsistencies between text and Table 1 (61.9 vs 62.8; “validation” vs “evaluation” set) noted above.

Use in our design. This is the recipe for *per-task test-time training* on the worker: when the router flags a task as novel and the task carries examples (failing tests, example I/O, a reference diff), build leave-one-out ICL instances from them, apply semantics-preserving augmentations (for code: identifier renaming, statement reordering where safe, formatting changes — our analogue of invertible grid transforms), train a throw-away QLoRA with $\mathcal{L}^{outputs}$, vote hierarchically over augmentation groups, then discard. Budget on a 12 GB GPU: expect minutes per task, so reserve it for the Thinking tier after cheaper routes fail; cap $\mathcal{D}_{TTT}$ at 250 examples as the paper does.

PDF: https://arxiv.org/pdf/2411.07279 — Code: https://github.com/ekinakyurek/marc

1.5 Transformer^2: Self-adaptive LLMs (Sun, Cetin, Tang, 2025) — arXiv:2501.06252

Problem and claim. Full or LoRA fine-tuning per task is expensive, static, and can overfit. Transformer^2 (Transformer-Squared) fine-tunes only a vector of scales on the singular values of each weight matrix (Singular Value Fine-tuning, SVF), trains these “expert vectors” with RL on task reward, and at inference uses a two-pass scheme: pass 1 identifies the task and chooses/mixes experts, pass 2 answers with the adapted weights. Claims: fewer parameters than LoRA with better accuracy; works across LLaMA-3 8B/70B, Mistral-7B and a vision-language model; experts transfer across models. ICLR 2025.

Headline results. Training tasks (Table 1; GSM8K / MBPP-Pro / ARC-Easy): LLaMA3-8B-Instruct 75.89/64.65/88.59; +LoRA 77.18/67.68/88.97; +SVF 79.15/66.67/89.56. Mistral-7B-Instruct-v0.3 42.83/49.50/81.65; +LoRA 44.66/51.52/81.19; +SVF 49.74/51.52/85.14. LLaMA3-70B-Instruct 85.29/80.81/89.10; +LoRA 77.26/68.69/88.55; +SVF 88.32/80.81/88.47. Unseen tasks (Table 2; MATH / HumanEval / ARC-Challenge): LLaMA3-8B 24.54/60.98/80.63; +LoRA 24.12/52.44/81.06; +T^2 Prompt 25.22/61.59/81.74; +T^2 Cls-expert 25.18/62.80/81.37; +T^2 Few-shot 25.47/62.99/82.61. Mistral-7B 13.02/43.29/71.76; +LoRA 13.16/37.80/75.77; +Prompt 11.86/43.90/72.35; +Cls-expert 11.60/43.90/74.83; +Few-shot 13.39/47.40/75.47. LLaMA3-70B 40.64/78.66/87.63; +LoRA 25.40/73.78/83.70; +Prompt 40.44/79.88/88.48. VLM (LLaMA3-LLaVA-NeXT-8B, TextVQA → OKVQA): SVF boosts the vision-language task by over 39% (Figure 5; exact values not in fetched text). Inference overhead of prompt-based pass 1 (Table 3; first / second pass seconds): MATH 42.64 (13%) / 321.19; HumanEval 2.76 (19%) / 14.28; ARC-Challenge 13.40 (47%) / 28.51.

Method — equations. (Section 3)

SVD: $W = U\Sigma V^\top$, $W \in \mathbb{R}^{n\times m}$, $\Sigma \in \mathbb{R}^{r\times r}$ diagonal with singular values in descending order, U, V semi-orthogonal; $y = \sum_{i=1}^{r} \sigma_i u_i v_i^\top x$.

SVF parametrisation: for each fine-tuned matrix learn $z \in \mathbb{R}^r$,

$$W' = U\Sigma' V^\top, \qquad \Sigma' = \Sigma \otimes \operatorname{diag}(z)$$

(elementwise scaling of the singular values; $\theta_z = \{z_1, ..., z_{N\times M}\}$ for N layers and M matrices per layer; $r = \min(m, n)$ parameters per matrix vs $(m+n)\,r'$ for LoRA.)

RL objective, Eq. (1):

$$J(\theta_z) = \mathbb{E}\left[\log\left(\pi_{\theta_{W'}}\left(\widehat{y}_i \mid x_i\right)\right) r\left(\widehat{y}_i, y_i\right)\right] - \lambda D_{KL}\left(\pi_{\theta_{W'}} \parallel \pi_{\theta_W}\right)$$

$\pi_{\theta_{W'}}$: model with W replaced by W'; $\widehat{y}_i$ sampled answer for prompt x_i, y_i reference; $r \in \{-1, 1\}$ binary correctness (-0.1 instead of -1 for VLM stability); λ small KL coefficient; optimiser REINFORCE.

Few-shot adaptation (CEM): $z' = \sum_{k=1}^{K} \alpha_k z_k$, $\alpha \in \mathbb{R}^K$ found by the cross-entropy method on 10 held-out samples: sample candidate α from a Gaussian, score on the held-out set, refit $\alpha \leftarrow \text{mean}(\text{elites})$, $\sigma \leftarrow \text{std}(\text{elites})$, up to 100 iterations; ties broken by highest average log-likelihood of the correct generated answers; variants per-vector (3 coefficients) or per-layer, normalised or not. Population size and elite fraction: (not in fetched text — verify in paper §3.2 / Appendix).

Method — algorithm.

```
offline (per task k): SVD every target matrix once; init z ~ N(0.1, 1e-3); sample answers with W'(z);
    REINFORCE on Eq. (1) with r in {-1,+1}; AdamW lr 2e-3 cosine, batch 256, grad clip, update at epoch end,
early stop.
inference (two passes):
  pass 1 (dispatch) — one of:
    (a) Prompt: ask the base model to classify the prompt into {math, code, reasoning, others}; others ->
base weights
    (b) Cls-expert: load a dedicated SVF expert z^c trained on D = {(x_{i,k}, k)} to classify
    (c) Few-shot: CEM over alpha on 10 held-out task samples -> z' = sum_k alpha_k z_k
  pass 2: answer with W' = U (Sigma * diag(z')) V^T
```

Hyperparameters and setup. SVF on MLP and attention matrices (half the layers for 70B and the VLM to save memory); $\lambda \in \{0.0, 0.1, 0.2, 0.3\}$ chosen on validation; LoRA baseline rank 16, α 32, dropout 0.05, on q and v projections, batch 256, lr swept 2e-6–5e-2. Parameter counts (LLaMA3-8B, Table 4): SVF MLP 0.39M, attention 0.16M, both 0.58M; LoRA attention 6.82M, MLP+attention 35.13M. A light 3-shot CEM variant with 10 generations finishes ARC-Challenge adaptation in about 11 minutes. Hardware/GPU-hours not stated.

Ablations that matter. (Table 4, LLaMA3-8B trained on GSM8K; GSM8K / MATH) base 75.89/24.54; SVF-RL MLP 78.62/24.20; SVF-RL attention 76.19/24.20; SVF-RL MLP+attention 79.23/25.04; SVF with next-token loss (attention) 60.50/18.52; LoRA-RL attention 57.92/15.72 (unstable, collapsed); LoRA next-token attention 77.18/24.12; LoRA next-token MLP+attention 75.66/22.12. Cross-model transfer (Table 5, LLaMA experts on Mistral; MATH/HumanEval/ARC-C): Mistral 13.02/43.29/71.76; +Llama SVF ordered 11.96/45.12/72.01; shuffled singular values 10.52/40.24/70.82; +few-shot cross-model 12.65/46.75/75.64.

Code / data / license. https://github.com/SakanaAI/self-adaptive-llms (Apache-2.0; `svd_reinforce_hydra.py`, `scripts/train_task_expert.sh`, `scripts/eval_few_shot.sh`). Data: GSM8K, MBPP-Pro, ARC-Easy, MATH, HumanEval, ARC-Challenge, TextVQA, OKVQA.

Stated limitations. “the capabilities of SVF experts are tied to the latent components of the base model”; CEM scaling to many domains raises the one-time adaptation cost; sparse rewards with a weak base model; cross-model transfer at different scales is open.

Use in our design. SVF expert vectors are a far smaller unit of stored skill than a LoRA (0.58M vs 35M params for an 8B model) and are RL-trained on exactly the binary reward our verifier produces, so they are the natural format for the P6 “skill vector” archive: one z per project/language/task family, mixed per task by CEM on a handful of the project’s own verified examples. For a 9B worker in 12 GB VRAM, the SVD factors cannot be held in full precision alongside the 4-bit base; store U, V once in fp16 on disk, apply z only to MLP matrices first (0.39M params, most of the gain in Table 4), and reconstruct W' per layer on load.

PDF: https://arxiv.org/pdf/2501.06252 — Code: https://github.com/SakanaAI/self-adaptive-llms

1.6 Nested Learning: The Illusion of Deep Learning Architectures (Behrouz, Razaviyayn, Zhong, Mirrokni, 2025) — arXiv:2512.24695

(NeurIPS 2025; Google Research blog post 7 Nov 2025; arXiv v1 31 Dec 2025. The arXiv HTML and the fetchable PDF text are truncated after §4.4; the NeurIPS camera-ready (OpenReview, 23 pp) supplied the CMS equations and Table 1. §8.1 self-referential Titans equations are **not in any fetched text**; what is given below for §8.1 is from a secondary source and is marked.)

Problem and claim. A model plus its optimiser is one system of *nested, multi-level optimisation problems*, each with its own context flow and update frequency. Under this view backpropagation, momentum and Adam are associative memories that compress gradients; attention/linear RNNs and Titans are 1- or 2-level in-context learners; and continual learning fails because standard models have too few levels. Contributions: (i) “deep

optimisers" (momentum with an L2-regression objective, MLP momentum, Delta-rule variants), (ii) a self-modifying sequence model that learns its own update rule, (iii) a Continuum Memory System (CMS) of MLP blocks at different update frequencies; combined into HOPE.

Headline results. (Table 1, camera-ready; Wiki ppl / LMB ppl / Avg acc over LMB-acc, PIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, SIQA, BoolQ) 760M / 30B tokens: Transformer++ 25.21/27.64/48.69; RetNet 26.08/24.45/48.46; DeltaNet 24.37/24.60/48.97; TTT 24.17/23.51/47.32; Samba (hybrid) 20.63/22.71/51.08; Titans (LMM) 20.04/21.96/51.56; HOPE 20.53/20.47/52.26. 1.3B / 100B tokens: Transformer++ 18.53/18.32/52.25; RetNet 19.08/17.27/52.02; DeltaNet 17.71/16.88/52.14; Samba 16.13/13.29/54.00; Titans (LMM) 15.60/11.41/56.82; HOPE 15.11/11.63/57.23. A HOPE row 26.05/29.38/46.90 appears for the 340M block (baselines for 340M not in fetched text). Continual learning: HOPE beats ICL, EWC and InCA on class-incremental tasks; "HOPE-3" nearly eliminates forgetting on a two-language continual task; NIAH/BABILong hold to about 10M tokens; MAD fuzzy recall HOPE 52.1 vs Transformer 47.9; formal-language recognition Transformer 0.0 vs HOPE 100 (these from secondary sources/blog — verify in paper §9).

Method — equations. (arXiv numbering unless marked CR = camera-ready)

Definition 1 (associative memory), Eq. (6): $\mathcal{M}^{*} = \arg\ \min_{\mathcal{M}} \tilde{\mathcal{L}}\left(\mathcal{M}\left(\mathcal{K}\right); \mathcal{V}\right)$ for keys $\mathcal{K}$, values $\mathcal{V}$.

One linear layer under GD as memory, Eqs. (7)–(9):

$$W_{t+1} = W_t - \eta_{t+1} \nabla_W \mathcal{L}\left(W_t; x_{t+1}\right) = W_t - \eta_{t+1} u_{t+1} \otimes x_{t+1}, \quad u_{t+1} := \nabla_{y_{t+1}} \mathcal{L}\left(W_t; x_{t+1}\right)$$

$$W_{t+1} = \arg\ \min_{W} \{\langle W x_{t+1}, u_{t+1}\rangle + \tfrac{1}{2\eta_{t+1}} \| W - W_t \|_2^2\}$$

u = local surprise signal (error in representation space). Definition 2: update frequency f_A = updates of component A per unit time (one data point = one unit); $A \succ B$ iff $f_A > f_B$, or $f_A = f_B$ and B's state at t needs A's state at t. Definition 3 (nested system of K levels), Eq. (19), level k problem i with objective $\mathcal{L}_i^{(k)}$, context $\mathcal{C}_i^{(k)}$:

$$\theta_{i,t+1}^{(k)} = \arg\ \min_{\Phi} \langle \Phi x_{t+1}, -\nabla \mathcal{L}_i^{(k)}\left(\theta_{i,t}^{(k)}; x_{t+1}\right)\rangle + \tfrac{1}{2\eta_{i,t+1}^{(k)}} \| \Phi - \theta_{i,t}^{(k)} \|_2^2$$

(Eq. 20: same with $\left(k_{t+1}^{(i)}, v_{t+1}^{(i)}\right)$ key-value contexts.)

Backprop as memory, Eqs. (29)–(31): $\partial \mathcal{L}/\partial W_\ell = \delta_\ell \widehat{x}_{\ell-1}^{\top}$, $\delta_\ell = J_{\phi_\ell}\left(z_\ell\right)^{\top}\left(W_{\ell+1}^{\top}\delta_{\ell+1}\right)$; $W_{\ell,t+1} = \arg\ \min_W \{\langle W \widehat{x}_{\ell-1}, \delta_\ell\rangle + \frac{1}{2\eta} \| W - W_{\ell,t} \|_F^2\}$.

Momentum as memory, Eqs. (33)–(34): $W_{\ell,t+1} = W_{\ell,t} + m_{\ell,t+1}$, $m_{\ell,t+1} = \alpha_{\ell,t+1} m_{\ell,t} - \eta_{\ell,t+1} \delta_\ell \widehat{x}_{\ell-1}^{\top}$; with $\alpha = 1$ momentum solves $\min_m \langle m \widehat{x}_{\ell-1}, \delta_\ell\rangle$ (dot-product similarity); $\alpha \neq 1$ adds an ℓ_2 regulariser. Generalised, Eqs. (35)–(37): $m_\ell = \arg\ \min_m \tilde{\mathcal{L}}\left(m; \widehat{x}_{\ell-1}, -\delta_\ell\right)$ with an internal objective $\tilde{\mathcal{L}}$.

More expressive momentum (§4.4), with $g_i = \nabla \mathcal{L}\left(W_i; x_i\right)$ and preconditioner P_i: Eq. (46)–(47) $\min_m \langle m g_i^{\top}, P_i\rangle$, $m_{i+1} = m_i - \eta_i P_i g_i$ (forget gate α_{i+1} stated in text, omitted in printed Eq. 47). Delta momentum (L2-regression internal objective), Eqs. (48)–(49):

$$W_{i+1} = W_i + m_{i+1}, \qquad m_{i+1} = m_i\left(\alpha_{i+1} - g_i^{\top} g_i\right) - \eta_t P_i g_i$$

(CR Eq. 22 writes $\left(\alpha_{i+1} I - g_i^{\top} g_i\right) m_i$.) Deep Momentum GD, Eq. (50): $W_{i+1} = W_i + m_{i+1}\left(u_i\right)$, $m_{i+1} = \alpha_{i+1} m_i - \eta_t \nabla \mathcal{L}^{(2)}\left(m_i; u_i, 1\right)$, $u_i = g_i$, m an MLP. Eq. (51): $\phi\left(g_i\right)$ higher-order feature map; Eq. (52): $W_{i+1} = W_i + \sigma\left(m_{i+1}\left(u_i\right)\right)$ — with σ = Newton–Schulz and linear m this is Muon.

L2-regression gradient descent used as HOPE's internal optimiser (CR Eqs. 27–29):

$$\min_W \| W x_t - \nabla_{y_t} \mathcal{L}\left(W_t; x_t\right) \|_2^2 \ \Rightarrow\ W_{t+1} = W_t\left(I - x_t x_t^{\top}\right) - \eta_{t+1} \nabla_{y_t} \mathcal{L}\left(W_t; x_t\right) \otimes x_t$$

("a new variant of gradient descent"; the name Delta Gradient Descent appears in the arXiv intro, not the camera-ready.)

Continuum Memory System (CR Eqs. 30–31; arXiv §7.1): chain of k MLP blocks with frequencies $f_1 > f_2 > \ldots > f_k$,

$$y_t = \mathrm{MLP}^{(f_k)}\left(\mathrm{MLP}^{(f_{k-1})}\left(\cdots \mathrm{MLP}^{(f_1)}\left(x_t\right)\right)\right)$$

$$\theta_{i+1}^{(f_\ell)} = \theta_i^{(f_\ell)} - \begin{cases} \sum_{t=i-C^{(\ell)}}^{i} \eta_t^{(\ell)} f(\theta_t^{(f_\ell)}; x_t) & \text{if } i \equiv 0 \pmod{C^{(\ell)}} \\ 0 & \text{otherwise} \end{cases}$$

$C^{(\ell)}$ chunk size of level ℓ (printed definition "$C^{(\ell)} := \max_\ell C^{(\ell)}/f_\ell$" is garbled in extraction — verify in paper §7.1); $f(\cdot)$ = the error term of any optimiser (e.g. $\nabla\mathcal{L}$ for GD). A conventional Transformer block is the special case $k = 1$.

Self-referential Titans (§8.1) — **from a secondary source citing Eqs. 76, 79, 83–85; verify in paper §8.1**: plain Titans has static projections ($\eta_t = x_t W_\eta$ etc., Eq. 76); the adaptive version gives each of $\mathbf{k}, \mathbf{v}, \eta, \alpha$ its own in-context memory, e.g. $\eta_t = \mathcal{M}_{\eta,t-1}(x_t)$ (Eq. 79); the self-modifying version has each memory generate its own target $\widehat{\mathbf{v}}_{\square,t} = \mathcal{M}_{\square,t-1}(\mathbf{v}_t)$, $\square \in \{\mathbf{k}, \mathbf{v}, \eta, \alpha, \text{memory}\}$ (Eqs. 83–85); $\mathbf{q}_t = x_t W_q$ stays non-adaptive; memories are 2-layer residual MLPs updated with the L2-regression/delta rule above plus retention gate α_t; initial states are meta-learned against the outer next-token loss.

Method — algorithm. (CMS schedule, from CR Eq. 31)

```
levels l = 1..k with chunk sizes C_1 < C_2 < ... < C_k  (f_l = 1/C_l), params theta_l, accumulators A_l = 0
for step i, input x_i:
    y = x_i ; for l in 1..k: y = MLP_l(y; theta_l)
    for l in 1..k: A_l += eta_l * err_l(theta_l; x_i)     # gradient (or any optimiser's error term)
    for l in 1..k: if i % C_l == 0: theta_l -= A_l ; A_l = 0
HOPE block = self-modifying Titans memory (per-token level) -> CMS stack (slower levels); pre-training =
slowest level.
```

Hyperparameters and setup. 340M, 760M (30B tokens), 1.3B (100B tokens); datasets, context length and optimiser deferred to Appendix G (not in fetched text). Secondary sources describe a 3-level CMS (periods on the order of 512 / 2,048 / 8,192 tokens) and HOPE-3 — unverified. M3 optimiser (CMS applied to momentum: fast momentum every step, slow every ~100) beats AdamW/Muon on ImageNet-21K ViT pre-training (numbers not in fetched text).

Ablations that matter. Text/secondary: gains over ICL come from multiple levels (CMS); gains over Titans come from self-reference + CMS; "Hope-Attention" (HOPE with attention) outperforms Transformer; removing CMS or self-modification degrades. Numbers: (not in fetched text — verify in paper §9.6).

Code / data / license. No code at fetch time (authors "plan to release data and code on GitHub"). Blog: https://research.google/blog/introducing-nested-learning-a-new-ml-paradigm-for-continual-learning. Camera-ready: https://openreview.net/pdf/9082b3fb3c37bdc2a3b5d69681382ebe783d49e3.pdf.

Stated limitations. Not stated in fetched text. Language-model experiments stop at 1.3B / 100B tokens; §8.1 details and all §9 tables beyond Table 1 are unverified here.

Use in our design. Nested Learning gives the *organising principle* for the whole P6 track: every learning mechanism in this pack is a level with an update frequency — Titans/TTT memory (per token/chunk), per-task LoRA/SVF (per task), SEAL self-edits and weekly LoRA (per day/week), base weights (frozen). Implement CMS literally as the scheduler: each level keeps a gradient accumulator and applies it every $C^{(\ell)}$ steps. Use the L2-regression (delta) rule $W \leftarrow W\left(I - xx^\top\right) - \eta\, u \otimes x$ as the memory write instead of plain Hebbian/Titans GD once the basic Titans memory works; the self-referential memories are research-grade and belong after that.

PDF: https://arxiv.org/pdf/2512.24695 — Code: none at fetch time

2. Design extraction for the module

2.1 Algorithms to implement (consolidated pseudocode)

A. Neural memory layer (Titans + TTT engineering), PyTorch. One module per attention head group; frozen backbone; MAG wiring.

```
class NeuralMemory(nn.Module):
    # outer-loop params: W_K, W_V, W_Q (d x d), 1-D depthwise conv on each, gate projections for alpha,
theta, eta,
    #                    memory init M_0 (2-layer MLP d -> 4d -> d with GELU, LN+residual wrapper), theta_lr
    # inner-loop state : M (copy of M_0 at sequence start), S (zeros like M)
    def forward(x, state):                          # x: [B, T, d], T a multiple of chunk b
        k = conv(x W_K); v = conv(x W_V); q = conv(x W_Q); k, q = l2norm(k), l2norm(q); apply SiLU per paper
§4.4
        alpha = sigmoid(x W_a); theta = theta_base * sigmoid(x W_t); eta = sigmoid(x W_e)   # functional
form: our choice
        for each chunk c (b tokens):
            M_c = state.M                             # chunk-start weights (TTT mini-batch rule)
            g_t = grad_{M_c} || M_c(k_t) - v_t ||^2  for all t in c   # one batched forward/backward (dual
form if linear)
            S_t = eta_t S_{t-1} - theta_t g_t          # linear recurrence, associative scan over the chunk
            beta ratios: M_t = prod_{j<=t}(1-alpha_j) M_c + sum_i (beta_t/beta_i) S_i-terms   # Eq. 16
            y_t = M_{t-1}(q_t)                        # retrieval at chunk resolution (M_c) or per token
(exact)
            state.M, state.S = M_end_of_chunk, S_end_of_chunk
        return y, state
block output (MAG): o = norm(sw_attn(P || x)) * sigma(norm(y))        # Eq. 28 gate
```

Training: outer loop = next-token loss, backprop through the inner updates (TTT §2), gradient checkpointing per chunk; start with memory-branch-only training (backbone frozen). Serialise `state` per project.

B. CMS scheduler (Nested Learning). Wrap every trainable level in `{params, accumulator, period C}`; each step accumulate the level's error term; apply and zero when `step % C == 0`. Levels in our system: L0 neural memory (C=chunk), L1 per-task adapter (C=task), L2 self-edit/weekly LoRA (C=day/week), L3 base (C=infinity).

C. Per-task test-time training (ARC TTT).

```
ttt(task):   # task has K >= 2 examples (tests/IO pairs/diffs) and a query
    D = [];  for j in 1..K: D += leave_one_out(examples, j)         # ICL format, not direct I/O
    D = augment(D, T)  (semantics-preserving, invertible)  + permute example order; cap 250
    lora = QLoRA(rank 128, alpha 16, modules q,v,mlp,out); train 2 epochs, bs 1-2, AdamW, lr 5e-5 (small) /
1e-4 (8B+)
           with loss on every example output + held-out output (L^outputs), not on inputs
    preds = { t^-1(greedy(worker+lora, t(examples), t(query))) for t in T, 2 example orders }
    per t: top-3 by frequency; global: top-2; ties -> identity first; return top-2 (verifier picks)
    discard lora (or archive if the verifier passes and the task recurs)
```

D. Self-edit outer loop (SEAL / ReST-EM) with the verifier as reward.

```
for round in 1..R (R = 2 to start):
    for each context C (task family with held-out verified cases tau), sample M self-edits SE ~ worker(. |
C, prompt)
        # SE = {synthetic examples, lora_rank, lr, epochs, loss_mask, augmentation flags}
        theta' = QLoRA_SFT(worker, SE); r = 1 if verifier(theta', tau) > verifier(worker, tau) else 0
    SFT worker-policy LoRA on {SE : r = 1} (rank 64, alpha 128, 2 epochs, lr 3e-4, bs 10)   # SEAL KI
settings
    archive every rewarded SE as a dataset; never chain edits on one weight set (forgetting)
```

E. SVF skill vectors (Transformer^2). Offline: SVD of each target matrix (MLP first); per skill, REINFORCE on z with verifier reward in {−1, +1}, KL to base, lr 2e-3, bs 256, init $z \sim \mathcal{N}\left(0.1, 10^{-3}\right)$. Online per task: CEM over α on 10 held-out verified examples (≤100 iterations; light variant 3-shot), $z' = \sum_k \alpha_k z_k$, reconstruct W' and run pass 2.

2.2 Equations and quantities the implementation must compute

- Associative loss $\ell = \| \mathcal{M}\left(\mathbf{k}_t\right) - \mathbf{v}_t \|_2^2$ and its gradient (surprise) at chunk-start weights $\mathcal{M}_{t'}$, $t' = t - \mathrm{mod}\left(t, b\right)$.
- Momentum/past-surprise recurrence $S_t = \eta_t S_{t-1} - \theta_t g_t$ (associative scan), forgetting $\mathcal{M}_t = \left(1 - \alpha_t\right)\mathcal{M}_{t-1} + S_t$, cumulative retention $\beta_t = \prod_{j \le t}\left(1 - \alpha_j\right)$.
- Retrieval $y_t = \mathcal{M}^*\left(\mathbf{q}_t\right)$ (no grad). Persistent tokens P prepended.
- Dual-form outputs for a linear memory: $W_b = W_0 - 2\eta\left(W_0 X - X\right)X^\top$, $Z = W_0 X - 2\eta\left(W_0 X - X\right)\,\mathrm{mask}\left(X^\top X\right)$; general MLP via the Appendix-A forward/backward/output

passes.

- Learnable inner lr $\eta(x) = \eta_{base}\sigma(\theta_{lr} \cdot x)$; $f(x) = x + \mathrm{LN}(f_{res}(x))$.
- Delta/L2-regression write (HOPE optimiser): $W_{t+1} = W_t\left(I - x_t x_t^\top\right) - \eta u_t \otimes x_t$.
- CMS: per-level accumulated update applied when $i \equiv 0 \pmod{C^{(\ell)}}$.
- TTT loss $\mathcal{L}^{outputs}$ (mask inputs; include every demonstration output from $k = 2$); hierarchical voting Top-3 per transform then Top-2 global.
- SEAL reward $r \in \{0, 1\}$ = improvement of the adapted model on τ; filtered-BC gradient (Eq. 4 with $r_{ij} \in \{0, 1\}$).
- SVF: $W' = U(\Sigma \otimes \mathrm{diag}(z))V^\top$; REINFORCE objective Eq. (1) with KL penalty λ; CEM mixing $z' = \sum_k \alpha_k z_k$.

2.3 Data structures and file formats (as TypeScript-style type sketches)

```
        type NeuralMemoryConfig = { dim: number; memDepth: 1|2|3|4; hiddenMult: number; chunk: number /*
b=16 start */;
          thetaBase: number; persistentTokens: number; wiring: "MAG"|"MAC"|"MAL"|"LMM"; heads: number };
        type MemoryState = { M: Float16Array[] /* per-layer weights */; S: Float16Array[] /* momentum, same
shapes */;
          tokensSeen: number; projectId: string; savedAt: string };          // serialised per project at
session end
        type CMSLevel = { name: "memory"|"task"|"selfedit"|"base"; period: number /* C^(l); Infinity =
frozen */;
          params: string /* adapter path */; accumulator?: string; lastApplied: number };
        type TTTTask = { id: string; examples: { input: string; output: string }[]; query: string;
          augmentations: ("rename"|"reorder"|"reformat"|"identity")[]; cap: 250; lora: { rank: 128; alpha:
16; lr: number; epochs: 2 } };
        type TTTVote = { byTransform: Record<string, { pred: string; count: number }[]>; final: [string,
string] };
        type SelfEdit = { contextId: string; data: { prompt: string; completion: string }[];
          hp: { rank: 32|64; alpha: 32|64; lr: 1e-4|3e-4|5e-4|1e-3|2e-3; epochs: 1|3|5; batch: 1|4|8|16;
lossMask: "all"|"outputs" };
          reward: 0|1; verifierScoreBefore: number; verifierScoreAfter: number; round: number };
        type SVFExpert = { name: string; baseModelHash: string; modules: ("mlp"|"attn")[];
          z: Record<string /* layer.matrix */, Float32Array /* length r */>; trainedOn: string; reward:
"verifier" };
        type SVFMix = { experts: string[]; alpha: number[]; heldOut: string[] /* 10 verified example ids */;
cemIters: number };
```

2.4 Hyperparameters and defaults to start from (with the paper each comes from)

Quantity	Default	Source
Inner mini-batch / chunk b	16	TTT layers (Table 1: largest single gain)
Memory depth $L_{\mathcal{M}}$	2 (MLP, hidden 4x, GELU)	Titans §5.5 (>=2 better); TTT-MLP
η_{base}	1.0 linear / 0.1 MLP, swept {0.01,0.1,1,10}	TTT layers §2.7
Inner f wrapper	$x + \mathrm{LN}(f_{res}(x))$	TTT layers (−1.22 ppl)
Gates $\alpha_t, \theta_t, \eta_t$	token-dependent (form unspecified)	Titans §3.1
Outer optimiser (memory pre-training)	AdamW lr 4e-4, wd 0.1, cosine, 0.5M-token batch	Titans §5.1
Per-task LoRA	r=128, α=16, 2 epochs, bs 1–2, lr 5e-5 / 1e-4, cap 250 examples	ARC TTT Table 6
Voting	top-3 per transform, top-2 global, identity wins ties, 2 orders per transform	ARC TTT App. E
Self-edits per context M	5 (knowledge) / 15 (few-shot), temp 1	SEAL §4
ReST-EM rounds	2	SEAL App. B
ReST-EM SFT LoRA	r=64, α=128, 2 epochs, lr 3e-4, bs 10	SEAL App. B
Inner self-edit LoRA search	r {32,64}, lr {1e-4..2e-3}, epochs {1,3,5}	SEAL Tables 3–4

Quantity	Default	Source
SVF training	REINFORCE, lr 2e-3 cosine, bs 256, z init N(0.1, 1e-3), λ in {0,0.1,0.2,0.3}, r in {−1,1}	Transformer^2 §4
SVF modules	MLP (+attention when memory allows)	Transformer^2 Table 4
CEM few-shot	10 held-out samples, <=100 iters; light: 3-shot, 10 generations	Transformer^2 §3.2, App. D
CMS periods	geometric ladder, Transformer = 1 level; secondary sources cite ~512/2048/8192 tokens	Nested Learning §7.1

2.5 Evaluation: how to know the module works (metrics, from the papers)

- Neural memory: perplexity vs context length must keep decreasing past 16k (TTT Figure 2 criterion; Mamba plateaus); S-NIAH pass-key/number/word retrieval at 2K–16K (Titans Table 2: >=95% at 16K for MAC); ablation checks — removing forgetting or momentum should cost >=2 ppl and >=5 long-context points (Titans Table 5).
- Per-task TTT: solved-task count before/after TTT on a held-out slice of our benchmark (ARC: 5→29 of 80 at 1B); leave-one-out vs direct I/O and per-task vs shared adapter deltas should reproduce in sign; wall-clock per task (paper: about 7 min/task on A100 at 8B).
- Self-edit loop: fraction of rewarded self-edits per round (ARC: 20% → 72.5% after RL); held-out accuracy of the adapted worker vs training on the raw context (SQuAD: 33.5 → 47.0); forgetting curve = accuracy on earlier contexts after k sequential edits (SEAL Figure 6).
- SVF: task accuracy of SVF vs LoRA on the training skill and on an unseen related task (Table 1/2 pattern); parameter count per skill; first-pass overhead ratio (Table 3: 13–47%).
- CMS/HOPE: continual-learning retention (accuracy on task A after training on B) vs a single-level baseline; MAD/NIAH-style recall.

2.6 Known pitfalls and failure modes (from the papers' limitations)

- Sequential weight edits cause gradual catastrophic forgetting (SEAL §5) — keep a frozen base and per-topic adapters; evaluate retention.
- Each self-edit evaluation is a full fine-tune + eval (30–45 s on A100 in SEAL; minutes on a 12 GB consumer GPU) — budget M and rounds; the loop needs labelled τ (our verifier supplies it; unlabelled corpora do not).
- TTT-MLP memory I/O makes wall-clock worse than FLOPs predict (TTT layers); start with TTT-Linear / shallow memory and measure.
- Inner-loop training without LN + residual and learnable W_0 is unstable (TTT Table 1).
- LoRA trained with RL collapsed in Transformer^2 (Table 4 row 5); use SVF for RL-trained skills, LoRA for SFT.
- SVF experts are bounded by the base model's singular components and are tied to one base checkpoint; shuffling singular-value order destroys transfer.
- Shared adapters hurt structured tasks (ARC −24%) but help BBH-style tasks — decide per task family.
- Titans gate forms, chunk size, window and N_p are unspecified; Nested Learning §8.1 equations and §9 tables are unverified here; no official Titans/HOPE code — the lucidrains implementation is unofficial.
- Chunk-level (vs token-level) gates lose expressiveness (Titans §3.2).

2.7 Open questions this section does not settle

- Whether a neural memory trained as a side branch on a frozen 9B backbone (MAG with backbone frozen) retains the paper's gains — all Titans/TTT results are from pre-training the whole model.
- The exact functional form of Titans' $\alpha_t, \theta_t, \eta_t$ and HOPE's self-referential memories (§8.1) — must be read from the full paper/code when released.
- The right invertible "augmentations" for code tasks (the ARC transforms are grid-specific) and whether per-task TTT on code reproduces the leave-one-out > direct-I/O ordering.
- How to assign CMS periods to our tiers and whether CMS-style accumulation of per-task adapter gradients (rather than discarding them) helps or hurts retention.

- SVF on a 4-bit quantised base: whether $U\Sigma'V^\top$ reconstruction per layer is acceptable in 12 GB VRAM, or whether z must be folded into a dequantise-on-load path.
- Reward design for self-edits beyond binary pass/fail (partial test pass rates, cost-aware rewards) — not studied in SEAL.

3. Verified source table

arXiv id	Title	Year	Venue	Code	PDF
2501.00663	Titans: Learning to Memorize at Test Time	2025	arXiv preprint	none official; unofficial https://github.com/lucidrains/titans-pytorch	https://arxiv.org/pdf/2501.00663
2407.04620	Learning to (Learn at Test Time): RNNs with Expressive Hidden States	2024	arXiv preprint (v4 Aug 2025)	https://github.com/test-time-training/ttt-lm-pytorch ; https://github.com/test-time-training/ttt-lm-jax	https://arxiv.org/pdf/2407.04620
2506.10943	Self-Adapting Language Models	2025	arXiv preprint (v2 Sep 2025)	https://github.com/Continual-Intelligence/SEAL	https://arxiv.org/pdf/2506.10943
2411.07279	The Surprising Effectiveness of Test-Time Training for Few-Shot Learning (v1: …for Abstract Reasoning)	2024	arXiv preprint (v2 Mar 2025)	https://github.com/ekinakyurek/marc ; https://github.com/adamzweiger/Fewshot-TTT	https://arxiv.org/pdf/2411.07279
2501.06252	Transformer^2: Self-adaptive LLMs	2025	ICLR 2025	https://github.com/SakanaAI/self-adaptive-llms	https://arxiv.org/pdf/2501.06252
2512.24695	Nested Learning: The Illusion of Deep Learning Architectures	2025	NeurIPS 2025	none at fetch time	https://arxiv.org/pdf/2512.24695